

Bootstrap — Topics 40-41: Utilities API & Customization (Advanced)

Study paper format: notes (Roman Urdu + English technical terms), mechanism explanation, + active recall Q&A at the end. Ye topics tab kaam ate hain jab Bootstrap ko CDN se use karne ki bajaye, npm/Sass se install kiya jaye — kyunke customization ke liye source files tak access chahiye hota hai.

40. Bootstrap Utilities API (Advanced)

Sirf tab zaroori jab aap customization karna chahte ho.

Ye kya hai?

Bootstrap 5 ki har utility class (jaise `m-3`, `text-primary`, `d-flex`) asal mein ek **central Sass map** (`$utilities`) se automatically generate hoti hai — ye classes manually har ek CSS file mein likhi hui nahi hain. Utilities API is `$utilities` map ko customize karne ka tarika hai.

Kyun use karte hain?

- Naye custom utility classes add karne ke liye (jo Bootstrap by default nahi deta)
- Existing utilities (jaise spacing scale) ko modify karne ke liye
- Un utilities ko **remove** karne ke liye jo project mein use nahi ho rahin (final CSS file ka size chota rakhne ke liye)

Basic Example (Sass file mein)

```
SCSS
```

```

$utilities: (
  "cursor": (
    property: cursor,
    values: auto pointer grab
  )
);

@import "bootstrap/scss/bootstrap";

```

Peeche ka mechanism: Ye `$utilities` map mein ek naya entry add karta hai. Compile hone par Bootstrap automatically classes generate kar dega: `cursor-auto`, `cursor-pointer`, `cursor-grab` — bilkul waise hi jaise built-in utilities (`text-*`, `bg-*`) generate hoti hain. Ye ek **loop-based Sass mechanism** hai — Bootstrap internally is map par iterate kar ke CSS rules banata hai.

Existing Utility Modify Karna

```

scss

$utilities: (
  "width": (
    property: width,
    class: w,
    values: (
      10: 10%,
      25: 25%,
      50: 50%,
      100: 100%,
    )
  )
);

```

Ye `w-10` jaisi extra value bhi add kar sakta hai default `w-25/50/75/100` set mein.

Note: Ye poora system CDN version mein use nahi ho sakta, kyunke CDN sirf **already-compiled CSS file** deta hai — usme koi Sass map nahi hoti jo customize ki ja sake. Isi liye ye topic npm/Sass installation ke sath directly juda hua hai.

41. Bootstrap Customization

Variables

Bootstrap ka poora design system (colors, spacing, fonts, breakpoints) **Sass variables** ke through control hota hai — inhe `_variables.scss` file mein define kiya gaya hai.

```
scss

$primary: #ff6600;
$border-radius: 0.75rem;
$font-family-base: 'Poppins', sans-serif;

@import "bootstrap/scss/bootstrap";
```

Peeche ka mechanism: Sass mein agar aap kisi variable ko `@import` se **pehle** dobara define kar do, to Sass automatically Bootstrap ki default value ki bajaye **aapki value** use karega (Sass variable overriding). Isliye order important hai — apni variables `@import "bootstrap"` se pehle likhni zaroori hain, warna Bootstrap ki default values already "lock" ho chuki hongi.

Sass

Bootstrap CSS mein nahi, **Sass (.scss)** mein likha gaya hai — jo phir compile ho kar normal CSS banta hai.

```
node_modules/bootstrap/scss/  
  _variables.scss      → sab default variables  
  _mixins.scss         → reusable style blocks (jaise media queries ke  
liye)  
  bootstrap.scss       → sab files ko import karne wali main file
```

Project mein apni khud ki `.scss` file banate hain (jaise `custom.scss`), usme variables override karte hain, phir Bootstrap import karte hain, aur ek Sass compiler (jaise `sass` npm package ya Vite ka built-in support) use kar ke ise final `.css` file mein compile karte hain.

```
bash  
  
npm install sass --save-dev
```

Kyun Sass zaroori hai customization ke liye? Plain CSS mein aap "variable change kar ke poore framework ko regenerate" nahi kar sakte — CSS variables (`--bs-primary`) runtime par kaam karte hain lekin structural cheezein (jaise kitne breakpoints hain, kitni utility classes generate hongi) sirf **compile-time Sass logic** se control hoti hain.

Theme Colors

Bootstrap ke default theme colors (`primary`, `secondary`, `success`, etc.) ek Sass map mein defined hote hain jise `$theme-colors` kehte hain.

```
scss
```

```
$theme-colors: (  
  "primary": #6610f2,  
  "success": #198754,  
  "brand": #ff6600  
);  
  
@import "bootstrap/scss/bootstrap";
```

Peeche ka mechanism: Isse Bootstrap automatically `btn-brand`, `text-brand`, `bg-brand` jaisi classes generate kar dega — kyunke `$theme-colors` map par hi loop chal kar sab color-based utility/component classes banti hain (buttons, alerts, badges, text colors, backgrounds — sab isi ek map se derive hote hain).

Custom Breakpoints

Default breakpoints (`sm`, `md`, `lg`, `xl`, `xxl`) bhi ek Sass map (`$grid-breakpoints`) mein defined hote hain.

SCSS

```
$grid-breakpoints: (  
  xs: 0,  
  sm: 600px,  
  md: 900px,  
  lg: 1200px,  
  xl: 1500px  
);  
  
$container-max-widths: (  
  sm: 560px,  
  md: 850px,  
  lg: 1140px,  
  xl: 1400px  
);  
  
@import "bootstrap/scss/bootstrap";
```

Peeche ka mechanism: Grid system (`col-md-*`), display utilities (`d-md-*`), aur container widths — sab isi `$grid-breakpoints` map se apni media query values lete hain. Map change karne se poora responsive system automatically naye breakpoints par shift ho jata hai — kahin bhi hardcoded `768px` dhoondh kar manually change nahi karna parta.

Custom Utilities

Ye Utilities API (Topic 40) ka hi practical use hai — apni khud ki naye utility classes add karna.

SCSS

```
$utilities: map-merge(  
  $utilities,  
  (  
    "letter-spacing": (  
      property: letter-spacing,  
      class: ls,  
      values: (  
        1: 1px,  
        2: 2px,  
        3: 3px,  
      )  
    )  
  )  
);  
  
@import "bootstrap/scss/bootstrap";
```

Note: `map-merge` use karna important hai (sirf `$utilities: (...)` likhne ki bajaye) agar aap Bootstrap ki existing utilities ko **poori tarah replace nahi**, balke unme apni add karna chahte ho.

Cheat Sheet

Topic	Key Concept
Utilities API	<code>\$utilities</code> Sass map — utility classes yahan se auto-generate hoti hain
Variables	<code>\$primary</code> , <code>\$border-radius</code> , etc. — <code>@import</code> se pehle override karo
Sass	Bootstrap <code>.scss</code> source files se banta hai, compile ho kar CSS banta hai
Theme Colors	<code>\$theme-colors</code> map — naya color add karne se <code>btn-*</code> , <code>text-*</code> , <code>bg-*</code> sab auto-generate
Breakpoints	<code>\$grid-breakpoints</code> + <code>\$container-max-widths</code> maps — poora responsive system yahan se control hota hai
Custom Utilities	<code>map-merge(\$utilities, (...))</code> se naye utility classes add karo bina existing ko replace kiye

Practice Task

1. Ek `custom.scss` file banao jisme `$primary` color change karo, aur ek naya `"brand"` theme color add karo `$theme-colors` map mein.
2. `$grid-breakpoints` modify kar ke ek custom `md` breakpoint (900px) set karo aur observe karo grid layout kaise shift hota hai.
3. Utilities API use kar ke ek naya `cursor-*` utility class set banao (`cursor-pointer`, `cursor-not-allowed`).

Active Recall — Utilities API & Customization

| Khud se answer karne ki koshish karo, phir neeche scroll karke check karo.

Q1. Bootstrap ki utility classes (jaise `m-3`, `text-primary`) manually likhi hoti hain ya kahin se generate hoti hain?

Q2. Utilities API kis Sass structure ko modify karta hai?

Q3. Utilities API CDN version ke sath kyun kaam nahi karta?

Q4. Sass variables override karne ke liye order kyun important hai — `@import` `"bootstrap"` se pehle ya baad mein apni variable define karni chahiye?

Q5. Bootstrap CSS mein direct likha gaya hai ya Sass mein? In dono mein farq kya hai?

Q6. `$theme-colors` map mein naya color add karne se kya automatically ho jata hai?

Q7. Custom breakpoints set karne ke liye konsi do Sass maps modify karni parti hain?

Q8. `$grid-breakpoints` change karne se sirf grid affect hota hai ya aur bhi cheezein?

Q9. Existing utilities ko replace kiye bina naya utility add karne ke liye konsa Sass function use karna chahiye?

Answers

<details> <summary>Answers dekhne ke liye click karein</summary>

A1. Ye kahin se generate hoti hain — ek central `$utilities` Sass map par loop chal kar automatically CSS classes generate hoti hain, manually har class ek-ek karke likhi hui nahi hoti.

A2. Ye `$utilities` Sass map ko modify karta hai — naye entries add kar ke naye utility classes generate ki ja sakti hain, ya existing entries modify kar ke unki values (jaise spacing scale) change ki ja sakti hain.

A3. Kyunke CDN sirf ek already-compiled, final CSS file deta hai — usme koi Sass map ya source logic nahi hoti jo modify ki ja sake. Customization ke liye Sass source files (npm install) tak access zaroori hai.

A4. Apni variable `@import "bootstrap"` se **pehle** define karni chahiye. Sass mein agar variable `@import` se pehle already defined ho, to Bootstrap us value ko use karega apni default ki bajaye — order ulta hone par default value already "lock" ho chuki hogi aur override kaam nahi karega.

A5. Bootstrap Sass (`.scss`) mein likha gaya hai, jo phir compile ho kar plain CSS banta hai. Farq ye hai ke Sass mein variables, maps, loops, aur mixins use ho sakte hain (customization aur code-reuse ke liye), jabke plain CSS mein ye structural features available nahi hote.

A6. Naya color add karne se Bootstrap automatically us color ke liye `btn-*`, `text-*`, `bg-*`, `alert-*`, `badge` jaisi sab related classes generate kar deta hai — kyunke ye sab components internally `$theme-colors` map par hi loop karte hain.

A7. `$grid-breakpoints` (breakpoint pixel values ke liye) aur `$container-max-widths` (har breakpoint par container ki max-width ke liye).

A8. Sirf grid nahi — display utilities (`d-md-*`), text alignment (`text-md-*`), aur container widths sab isi `$grid-breakpoints` map se apni media queries derive karte hain. Isliye ise change karne se poora responsive system affect hota hai.

A9. `map-merge($utilities, (...))` use karna chahiye — ye naye entries ko existing `$utilities` map ke sath merge kar deta hai, poore map ko replace kiye bina.

</details>